corporatetraveldc-dispatch

Platform Compatibility Reference

Legend	**YES**	Fully supported	**WSL2**	Works inside WSL2 on Windows	**SRC**	Source build required
	PART	Partial / workaround needed	**WEB**	Browser access only	**NO**	Not supported

Feature Matrix

Feature	Linux x86_64	Linux ARM64 (Pi 5)	macOS Apple Si	macOS Intel	Windows x64 (WSL2)	Android ARM64 (Termux)	iOS / iPadOS
Python services (FastAPI / uvicorn)	YES	YES	YES	YES	WSL2	YES	NO
Rootless Podman containers	YES	YES	PART	PART	PART	NO	NO
systemd Quadlets (auto-start)	YES	YES	NO	NO	WSL2	NO	NO
Local Ollama LLM inference	YES	YES	YES	YES	YES	PART	NO
REST feed polling (METAR / NWS / TFR)	YES	YES	YES	YES	WSL2	YES	NO
FAA SWIM push ingest (solace)	YES	SRC	NO	NO	WSL2	NO	NO
lxml XML parsing	YES	YES	YES	YES	WSL2	SRC	NO
ntfy push alerts	YES	YES	YES	YES	WSL2	YES	NO
Dispatch web UI (browser access)	YES	YES	YES	YES	YES	YES	WEB
SSE live event stream	YES	YES	YES	YES	WSL2	YES	WEB
Tailscale access	YES	YES	YES	YES	YES	YES	YES
CUI / audit log (on-device)	YES	YES	NO	NO	PART	PART	NO

Per-Platform Details

Linux x86_64 (AMD64)

Full support — recommended for non-Pi deployments

WORKS	Full Python service stack (web, poller, pusher, ingest containers)
WORKS	Rootless Podman with systemd Quadlet auto-start

WORKS	Ollama with all six recommended models (llama3.2:3b through gemma2:9b)
WORKS	FAA SWIM NMS push ingest — solace-pubsubplus prebuilt wheel available
WORKS	lxml, pydantic-core — prebuilt wheels, no source build needed
WORKS	All REST feed polling, ntfy push, CUI audit log
NOTE	*Preferred OS: Fedora. Also tested: Debian/Ubuntu, Arch.*
NOTE	*GPU inference (CUDA/ROCm) available if a compatible GPU is present — set OLLAMA_GPU=cuda in dispatch.env.*

Linux ARM64 (aarch64) — Raspberry Pi 5, SBCs

Full support — primary production deployment target

WORKS	Full Python service stack including all four containers
WORKS	Rootless Podman with systemd Quadlets
WORKS	Ollama: llama3.2:3b + mistral fit comfortably in 16 GB RAM; phi3.5 for 8 GB
WORKS	All REST feeds, ntfy push, CUI audit log
WORKS	lxml — ARM64 wheel available on PyPI
WORKS	pydantic-core — ARM64 wheel available
NOTE	*solace-pubsubplus (FAA SWIM ingest): no official ARM64 wheel. Must be built from source with gcc and the Solace C API headers. Documented in the SWIM setup guide. REST poll fallback covers all SWIM feeds automatically until credentials land.*
NOTE	*Ollama models larger than 7B require 12+ GB free RAM; gemma2:9b and qwen2.5:7b are feasible but leave less headroom for containers.*
NOTE	*Current Pi 5 (16 GB) runs llama3.2:3b + mistral simultaneously via OLLAMA_MAX_LOADED_MODELS=2.*

macOS Apple Silicon (arm64) — M1/M2/M3/M4

Full stack except SWIM ingest — excellent for development

WORKS	Full Python service stack (bare Python or via Podman Machine / Docker Desktop)
WORKS	Ollama.app with Metal GPU acceleration — fastest local inference of any platform
WORKS	All REST feed polling, ntfy push, Dispatch Drawer
WORKS	lxml, pydantic-core — Apple Silicon wheels available
WORKS	Tailscale client available (native .app)
NOTE	*solace-pubsubplus is Linux-native (Solace C library). Not available on macOS. SWIM push ingest requires running in a Linux VM or forwarding from a Pi. REST poll fallback is automatic.*

NOTE	*systemd Quadlets not available. Use launchd plists or run services manually for development.*
NOTE	*Podman Machine and Docker Desktop both work; Podman is preferred to stay consistent with Pi deployment.*
NOTE	*CUI audit log works in dev mode; production CUI handling should stay on the Pi.*

macOS Intel (x86_64)

Full stack except SWIM ingest — same constraints as Apple Silicon

WORKS	Same as Apple Silicon with x86_64 Python wheels
WORKS	Ollama with CPU inference (no Metal GPU on Intel Mac)
WORKS	All REST feeds, ntfy, Dispatch Drawer
NOTE	*Same SWIM / solace-pubsubplus limitation as Apple Silicon.*
NOTE	*Ollama on Intel Mac is CPU-only — inference is slower. llama3.2:3b is usable; mistral:7b is noticeably slower than on Apple Silicon or a GPU-equipped Linux host.*
NOTE	*Rosetta 2 not needed — native x86_64 binaries available for all packages.*

Windows x64 (via WSL2)

Full stack inside WSL2 — Ollama runs natively on Windows

WORKS	Full Python service stack inside WSL2 (Ubuntu 24.04 or Fedora recommended)
WORKS	Ollama runs natively on Windows and is reachable from WSL2 at the host IP
WORKS	SWIM ingest: solace-pubsubplus works inside WSL2 (Linux x86_64 environment)
WORKS	All REST feeds, ntfy, Dispatch Drawer
WORKS	Docker Desktop or Podman Desktop for container support
NOTE	*systemd in WSL2: systemd is available in WSL2 (Ubuntu 22.04+ with systemd=true in /etc/wsl.conf). Quadlets work if Podman is installed in WSL2.*
NOTE	*Ollama GPU: if the host has an NVIDIA GPU with WSL2 CUDA support, Ollama on Windows will use it automatically.*
NOTE	*Set OLLAMA_BASE_URL in dispatch.env to http://:11434 (use 'ip route show | awk /default/ {print $3}' from WSL2).*
NOTE	*install-windows.ps1 handles WSL2 setup, Ollama install, and runs install.sh inside WSL2 automatically.*

Android ARM64 (Termux on tablet / phone)

Bare Python mode — no containers; good for tablet kiosk

WORKS	Full Python service stack as bare processes (FastAPI, uvicorn, poller, pusher)
WORKS	All REST feed polling (METAR, NWS, TFR, ATCSCC, Amtrak, NOTAMs)
WORKS	ntfy push alerts
WORKS	Ollama for Android ARM64 (via Termux package or official ARM64 binary)

WORKS	llama3.2:3b (2.0 GB) and phi3.5 (2.2 GB) — recommended for constrained storage
WORKS	Dispatch web UI accessible from device browser
WORKS	Termux:Boot for auto-start on reboot
NOTE	